RAPPORT DE STAGE

Nom : BLANIC Clément

Formation : BTS SIO2

Entreprise : ENEDIS

Adresse : 32 Avenue de l'Europe 92400 Courbevoie

Stage du : 5 janvier 2026 au 6 février 2026

Maître de stage : ILCINKAS Chrystelle

Tuteur de stage : BENYOUSSEF Redouane

Sommaire

Introduction 2
Présentation de l'entreprise 3
1.1 Enedis et L'équipe de recette 3
1.2 L'environnement de travail 3
Cahier des Charges 5
2.1 Le besoin métier 5
2.2 Problématique existante (Le constat) 5
2.3 Mon rôle 6
Travail accompli 7
3. DÉVELOPPEMENT ET IMPLÉMENTATION JAVA 7
3.1. Architecture de la classe SqlWithValidationAction 7
3.2. Algorithme de traitement (Méthode execute) 8
4. STRATÉGIE DE TESTS ET VALIDATION 9
4.1. Analyse du test unitaire 9
4.2. Le Résultat 10
5. VALIDATION FONCTIONNELLE ET RÉSULTATS (CHUTNEY) 10
5.1. Test sur Chutney en local 10
6. LISTAGE ET ANALYSE DES ERREURS 12
6.1 Test sur Chutney en local 12
6.2 Analyse d'un cas concret 12
7. DÉPLOIEMENT ET INTÉGRATION CONTINUE (CI/CD) 13
8. INTÉGRATION ET INDUSTRIALISATION (PROJET RTL-AT) 15
8.1 L'adaptation en Kotlin 15
8.2 Déploiement massif (Le fichier sql.kt) 15
8.3 Résultat final en production 16
Conclusion 17
Annexes 18
Annexe 1 : Schéma du système Informatique 18
Annexe 2 : Exemple résultat d'un scenario sur Chutney 18

Introduction

As part of my academic preparation, I finished a technical internship at Enedis, a significant player in the French electrical distribution system. In order to create automation tools for the Linky smart meter ecosystem, I joined a cross-functional development team.

The goal of my internship was to automate repetitive tasks described in user support tickets, specifically by enhancing the **Chutney** testing framework. My main objective was to develop intelligent automated scripts capable of validating data retrieved from SQL queries.

Beyond simple data verification, I focused on creating a logic of **error interpretation**: my scripts were designed to analyze raw database statuses (such as failure or rejection codes) and translate them into clear, intelligible messages for the support teams. To ensure the reliability of these tools before their deployment, I implemented a rigorous testing strategy using **parameterized tests**. This involved injecting various datasets into my scripts to verify their behavior and robustness in different scenarios. Among the tools and technologies I worked with were **Kotlin/Java**, **Chutney**, **DBeaver**, and **IntelliJ IDEA**.

This report is structured as follows:

- The **first chapter** presents the **business context** and the **technical environment** in which I worked.
- The **second chapter** outlines the **objectives of the internship**, based on the department's needs and my supervisor's expectations.
- The **third chapter** details the **work accomplished**, including the development of automation scenarios, the tools used, and the challenges encountered during implementation.
- Finally, I reflect on the **outcomes of this experience**, both technically and professionally.

This internship allowed me to deepen my programming skills in a real-world context and to better understand the industrial challenges of large-scale automation in the energy sector.

Présentation de l'entreprise

1.1 Enedis et L'équipe de recette

Enedis, fondée en 2008, dans le contexte de l'ouverture du marché de l'électricité à la concurrence, est une entreprise de distribution d'électricité et gestionnaire du réseau électrique, spécialisée dans l'exploitation et la modernisation du réseau électrique, l'accompagnement des projets et collectivités locales. Elle assure l'équilibre du système électrique, propose un dépannage 24/24H, 7/7j et l'intervention clientèle, la gestion du comptage des consommations pour les fournisseurs d'énergies et enfin le raccordement aux réseaux des clients.

J'ai pu faire mon stage dans le SI Chaines Communicantes, dans l'équipe de recette transverse qui réalise les tests de validation et de cohérence de bout en bout sur le système Linky avec l'ensemble des partenaires

1.2 L'environnement de travail

Annexe 1 : Schéma du système Informatique

Lors de ce stage, j'ai intégré une équipe chargée de développer des programmes permettant d'automatiser certaines actions liées au compteur Linky, comme la modification de la puissance d'un compteur, et autres actions.

Technologies, outils et langages utilisés :

- **Langage :** Java
- **Automatisation :** Chutney
- **Formats de données :** JSON, XML
- **Environnement de développement :** IntelliJ IDEA
- **Base de données :** DBeaver
- **Sécurité / Connexion :** FortiClient VPN, clé USB sécurisé contenant un certificat nominatif
- **Applications internes :** LinkySup, Idatha Log

Durant ce stage j'ai pu utiliser différents outils de développement :

Java est un langage de programmation très répandu, apprécié pour sa fiabilité et sa capacité à fonctionner sur n'importe quel système (grâce à la JVM). Pour ce projet, nous avons utilisé IntelliJ IDEA de JetBrains. C'est l'environnement de développement de référence pour Java : il m'a permis de coder plus efficacement grâce à ses suggestions automatiques et ses outils de correction d'erreurs.

Chutney est un outil d'automatisation open source développé par Enedis. Il permet d'enchaîner des scénarios complexes de manière simplifiée. Grâce à Chutney, nous pouvons suivre en détail l'exécution des tests, y compris les éventuelles erreurs, ce qui facilite grandement le débogage et la validation des scénarios automatisés.

Annexe 2 : Exemple résultat d'un scenario sur Chutney

DBeaver est un outil de gestion de bases de données, utilisé pour interagir avec les données via le langage SQL. Il permet de consulter, modifier ou interroger facilement les bases de données.

IdathaLog est une plateforme web interne à l'entreprise, accessible uniquement via une connexion sécurisée grâce à un VPN (FortiClient) et une clé USB PKI (OpenTrust CMS). Ce site permet de consulter les différents logs générés lors de l'exécution des scénarios automatisés avec Chutney.

LinkySup est une application interne utilisée pour détecter et diagnostiquer les dysfonctionnements du système Linky. Il permet également d'administrer le parc des compteurs et des concentrateurs et de les superviser.

Bitbucket est un outil de gestion de code basé sur Git, similaire à GitHub. Il est utilisé en équipe pour gérer les versions du code, faciliter la collaboration et intégrer de nouvelles fonctionnalités dans les projets.

D'autres formats et langages comme **XML** et **JSON** sont également utilisés. Ils sont souvent générés automatiquement par IntelliJ IDEA et Chutney pour permettre le bon fonctionnement des programmes et des scénarios d'automatisation.

Dans les locaux où j'ai réalisé mon stage, j'étais installé à proximité des **salles de tests**, spécialement aménagées pour effectuer des essais sur les équipements liés au réseau électrique intelligent.

<u>Images de la salle des tests :</u>

Ces salles permettent de **simuler et reproduire des scénarios réels** dans un environnement maîtrisé, afin de tester le comportement des équipements, valider les développements ou diagnostiquer des dysfonctionnements.

Cahier des Charges

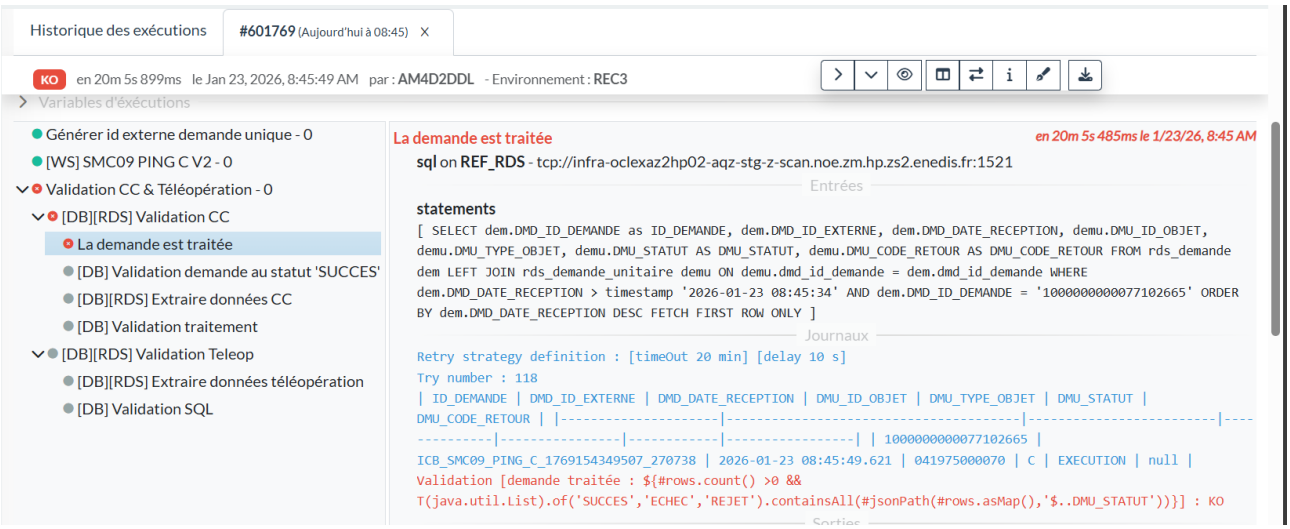
2.1 Le besoin métier

Ma mission principale consistait à concevoir et développer une solution d'automatisation pour faciliter l'analyse des données par les équipes de support. L'objectif était de remplacer les vérifications manuelles fastidieuses par un script intelligent capable d'interroger les bases de données et d'interpréter les résultats de manière autonome.

2.2 Problématique existante (Le constat)

Concrètement, les équipes faisaient face à des messages d'erreur techniques et illisibles, comme on peut le voir ci-dessous. Il était impossible de savoir rapidement si l'échec venait d'un problème de connexion, d'une donnée manquante ou d'une règle métier non respectée.

Voici l'erreur en question :


Historique des exécutions
#601769 (Aujourd'hui à 08:45)
KO en 20m 5s 899ms le Jan 23, 2026, 8:45:49 AM par : AM4D2DDL - Environnement : REC3
Variables d'éxécutions
Générer id externe demande unique - 0
[WS] SMC09 PING C V2 - 0
Validation CC & Téléopération - 0
[DB][RDS] Validation CC
La demande est traitée
[DB] Validation demande au statut 'SUCCES'
[DB][RDS] Extraire données CC
[DB] Validation traitement
[DB][RDS] Validation Teleop
[DB][RDS] Extraire données téléopération
[DB] Validation SQL
La demande est traitée
en 20m 5s 485ms le 1/23/26, 8:45 AM
sql on REF_RDS - tcp://infra-oclexaz2hp02-aqz-stg-z-scan.noe.zm.hp.zs2.enedis.fr:1521
Entrées
statements
[SELECT dem.DMD_ID_DEMANDE as ID_DEMANDE, dem.DMD_ID_EXTERNE, dem.DMD_DATE_RECEPTION, demu.DMU_ID_OBJET, demu.DMU_TYPE_OBJET, demu.DMU_STATUT AS DMU_STATUT, demu.DMU_CODE_RETOUR AS DMU_CODE_RETOUR FROM rds_demande dem LEFT JOIN rds_demande_unitaire demu ON demu.dmd_id_demande = dem.dmd_id_demande WHERE dem.DMD_DATE_RECEPTION > timestamp '2026-01-23 08:45:34' AND dem.DMD_ID_DEMANDE = '1000000000077102665' ORDER BY dem.DMD_DATE_RECEPTION DESC FETCH FIRST ROW ONLY]
Journaux
Retry strategy definition : [timeOut 20 min] [delay 10 s]
Try number : 118
| ID_DEMANDE | DMD_ID_EXTERNE | DMD_DATE_RECEPTION | DMU_ID_OBJET | DMU_TYPE_OBJET | DMU_STATUT | DMU_CODE_RETOUR | |---|---|---|

2.3 Mon rôle

Mon rôle était de concevoir et de programmer un module de validation capable d’analyser les retours de base de données et de traduire automatiquement les codes d'erreurs techniques. Une fois le script terminé et fiabilisé par des jeux de tests, il pouvait être utilisé quotidiennement pour fournir un diagnostic clair et immédiat à l’équipe recette, sans qu’un utilisateur ait besoin de décrypter manuellement des logs complexes.

Travail accompli

3. DÉVELOPPEMENT ET IMPLÉMENTATION JAVA

3.1. Architecture de la classe SqlWithValidationAction

Ci-dessous, nous retrouvons la première partie de la classe SqlWithValidationAction : le **constructeur**.

C'est une étape cruciale qui permet d'initialiser l'instance de l'action en lui injectant les composants techniques nécessaires à son exécution :

- **Le Logger :** Il permettra d'écrire dans les rapports d'exécution (pour que l'utilisateur voie ce qu'il se passe).
- **La Target :** Elle contient les informations de connexion à la base de données (URL, utilisateur, mot de passe).
- **Les Inputs :** Ce sont les paramètres entrés par l'utilisateur (la requête SQL à jouer et les validations attendues).

Ce constructeur garantit que l'action ne démarre pas avec des données manquantes.

```java
public class SqlWithValidationAction implements Action {  3 usages  CB4A4C2L
    private static final String CONFIGURABLE_NB_LOGGED_ROW = "chutney.actions.sql.max-logged-rows";  1 usage
    private static final Integer DEFAULT_NB_LOGGED_ROW = 30;  1 usage
    private static final String CONFIGURABLE_MINIMUM_MEMORY_PERCENTAGE_REQUIRED = "chutney.actions.sql.minimum-memory-percentage-required";  1 usage
    private static final Integer MINIMUM_MEMORY_PERCENTAGE_REQUIRED = 0;  1 usage

    private final Map<String, Object> validations;  2 usages
    private final Logger logger;  4 usages
    private final List<String> statements;  2 usages
    private final Target target;  2 usages
    private final ActionsConfiguration configuration;  2 usages
    private final Integer nbLoggedRow;  2 usages
    private final Integer minimumMemoryPercentageRequired;  2 usages
    private SqlAction sqlAction;  4 usages


    public SqlWithValidationAction(SqlAction sqlAction, Target target, Logger logger, ActionsConfiguration configuration, @Input("statements") List<String> sta
        this.statements = statements;
        this.logger = logger;
        this.validations = validations;
        this.target = target;
        this.configuration = configuration;
        this.nbLoggedRow = ofNullable(nbLoggedRow)
            .orElse(configuration.getInteger(CONFIGURABLE_NB_LOGGED_ROW, DEFAULT_NB_LOGGED_ROW));
        this.minimumMemoryPercentageRequired = ofNullable(minimumMemoryPercentageRequired)
            .orElse(configuration.getInteger(CONFIGURABLE_MINIMUM_MEMORY_PERCENTAGE_REQUIRED, MINIMUM_MEMORY_PERCENTAGE_REQUIRED));
        this.sqlAction = sqlAction;
    }
```

3.2. Algorithme de traitement (Méthode execute)

Ensuite, nous retrouvons le cœur du programme : la méthode ActionExectionResult execute(). Cette partie assure le traitement des données étape par étape :

- **La boucle sur les validations :** Le code parcourt chaque règle de validation demandée (validations.entrySet()). Pour chaque colonne à vérifier, on lance une analyse.
- **La liste validation :** Cette liste va avoir en clé le nom de la colonne du statut et en valeur les statuts attendus.
- **Le filtrage via Stream :** À l'intérieur de la boucle, je crée une liste temporaire (list) via un Stream. L'instruction .filter() permet de ne **garder** dans cette liste **que les valeurs qui sont différentes** de ce qui est **attendu**. En clair, on isole les erreurs potentielles.
- **La condition de vérification :** Je vérifie ensuite le contenu de cette liste temporaire :
    - **Si la requête SQL ne retourne aucune ligne :** j'injecte le message formaté dans la liste errorsList.
    - **Si après avoir passé le premier if la liste temporaire est vide :** Cela veut dire que tout correspond à l'attendu. Je loggue un succès (logger.info).
    - **Si elle n'est pas vide :** C'est là qu'on entre dans le traitement d'erreur. J'analyse la valeur avec la Regex (Rejet ou Echec) et j'injecte le message formaté dans la liste principale errorsList.
- **L'affichage final des erreurs :** Une fois que toutes les validations sont passées, je fais une dernière boucle sur errorsList pour afficher (logger.error), tous les messages d'erreurs accumulés.

```java
@Override  CB4A4C2L
public ActionExecutionResult execute() {
    sqlAction = new SqlAction(target, logger, configuration, statements, nbLoggedRow, minimumMemoryPercentageRequired);
    ActionExecutionResult sqlExecResult = sqlAction.execute();

    Rows rows = (Rows) sqlExecResult.outputs.get("rows");
    List<String> errorsList = new ArrayList<>();
    if (rows.count() == 0) {
        errorsList.add("La requête SQL n'a retourné aucune ligne");
        sqlExecResult = ActionExecutionResult.ko();
    } else {
        for (Map.Entry<String, Object> entry : validations.entrySet()) {
            List<String> list = rows.get(entry.getKey()) List<Object>
                .stream() Stream<Object>
                .map(Object::toString) Stream<String>
                .filter( String s -> !Arrays.stream(entry.getValue().toString().split( regex: ",")) Stream<String>
                    .toList() List<String>
                    .contains(s))
                .toList();

            if (list.isEmpty()) {
                logger.info("Validation [" + entry.getKey() + "] - attendue: " + entry.getValue());
            } else {
                if (Pattern.matches( regex: "^(REJ[0-9]{3}|ECH[0-9]{3})$", list.getFirst())) {
                    errorsList.addFirst( e: list.getFirst() + " - " + getErrorMessage(list.getFirst()) + ", attendue: " + entry.getValue());
                    sqlExecResult = ActionExecutionResult.ko();
                } else {
                    errorsList.add("Validation [" + entry.getKey() + "] - actuelle: " + list.getFirst() + ", attendue: " + entry.getValue());
                    sqlExecResult = ActionExecutionResult.ko();
                }
            }
        }
    }

    for (String msg : errorsList) {
        logger.error(msg);
    }

    return sqlExecResult;
```

4. STRATÉGIE DE TESTS ET VALIDATION

4.1. Analyse du test unitaire

Maintenant, passons au test unitaire. Voici la version finale, améliorée itérativement pour couvrir plusieurs cas de figure (succès, échec simple, rejet technique).

```
class SqlWithValidationActionTest {  CB4A4C2L
    SqlAction sqlActionMock = mock(SqlAction.class);  2 usages
    ActionsConfiguration configMock = mock(ActionsConfiguration.class);  1 usage
    Target target = TestTarget.TestTargetBuilder.builder()  1 usage
        .build();

    @Test  CB4A4C2L
    void multiValidation() {
        Column col = new Column( name: "DMU_STATUT",  index: 0);
        Column col2 = new Column( name: "STATUT",  index: 1);
        Column col3 = new Column( name: "ST",  index: 2);
        Rows rows = new Rows(List.of(
            new Row(List.of(
                new Cell(col,  value: "SUCCES"),
                new Cell(col2,  value: "REJ006"),
                new Cell(col3,  value: 1)

            ))

        ));
        Map<String, Object> validations = Map.of( k1: "DMU_STATUT",  v1: "SUCCES,ECHEC,REJET",  k2: "STATUT",  v2: "SUCCES",  k3: "ST",  v3: 0);
        DelegateLogger logger = new DelegateLogger();
        when(sqlActionMock.execute()).thenReturn(ActionExecutionResult.ok(Map.of( k1: "rows", rows)));
        SqlWithValidationAction action = new SqlWithValidationAction(sqlActionMock, target, logger, configMock, List.of(), validations,  nbLoggedRow: 5,  minimumMemoryPercentageRequired: 0);
        ActionExecutionResult result = action.execute();
        Assertions.assertTrue(result.outputs.containsKey("rows"));
        Assertions.assertEquals( expected: "Validation DMU_STATUT - attendu:SUCCES,ECHEC,REJET", logger.info.getFirst());
        Assertions.assertEquals( expected: "REJ006 - " + getErrorMessage( errorCode: "REJ006") + ", attendu:SUCCES", logger.errors.get(0));
        Assertions.assertEquals( expected: "Validation ST - actuelle: 1, attendu:0", logger.errors.get(1));
    }

}
```

Dans ce code, nous retrouvons trois étapes clés pour simuler un comportement réel sans avoir besoin de se connecter à une vraie base de données :

- **La simulation des données SQL (Rows, Column, Cell) :** Puisque c'est un test unitaire, je dois "mocker" (simuler) la réponse de la base de données.
    - Je définis d'abord les colonnes (Column) que la requête est censée renvoyer (ex: DMU_STATUT, STATUT).
    - Ensuite, je construis manuellement une ligne de résultat (Row) remplie de cellules (Cell) avec des valeurs arbitraires. Ici, j'injecte volontairement un cas mixte : une valeur correcte ("SUCCES"), une erreur critique ("REJ006") et une erreur de valeur ("1").
- **La définition des règles de validation (validations) :** Cette Map représente ce que l'utilisateur attendrait normalement. Par exemple, pour la colonne STATUT, j'indique que j'attends la valeur "SUCCES". C'est cette règle qui va être confrontée à mon "REJ006" simulé plus haut.
- **Les vérifications finales (Assertions) :** Une fois l'action exécutée, j'utilise des assertions pour vérifier que mon programme a réagi comme prévu :
    - **Assertions.assertEquals (Cas passant) :** Je vérifie que le premier log est bien un message de succès (car DMU_STATUT valait "SUCCES").

- **Assertions.assertEquals (Cas d'erreur traduite) :** C'est la ligne la plus importante. Je vérifie que pour le code "REJ006", le programme a bien généré un message d'erreur contenant la traduction (via getErrorMessage) et non juste le code brut.
- **Assertions.assertEquals (Cas d'erreur simple) :** Je vérifie enfin que pour la colonne ST, le programme remonte bien l'écart entre la valeur actuelle (1) et l'attendu (0).
- Dans ces lignes assertEquals, il faut noter l'ordre des paramètres : à gauche, on place toujours le résultat attendu (*Expected*), et à droite, le résultat réel renvoyé par le code (*Actual*).

4.2. Le Résultat

```
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
15:38:08.003 [main] INFO fr.enedis.iceberg.action.assertion.DelegateLogger -- Validation DMU_STATUT - attendu:SUCCES,ECHEC,REJET
15:38:08.006 [main] ERROR fr.enedis.iceberg.action.assertion.DelegateLogger -- REJ006 - Non conformité du message, attendu:SUCCES
15:38:08.006 [main] ERROR fr.enedis.iceberg.action.assertion.DelegateLogger -- Validation ST - actuelle: 1, attendu:0

Process finished with exit code 0
```

Ces 3 lignes représentent les 3 vérifications (assertEquals) faites dans le test.

Comme prévu, on voit que les erreurs sont bien traduites. Cela confirme que j'ai bien testé les 3 cas possibles (le succès, l'erreur traduite et l'erreur simple) lorsque des données sont injectées dans le programme.

5. VALIDATION FONCTIONNELLE ET RÉSULTATS (CHUTNEY)

5.1. Test sur Chutney en local

Une fois le développement terminé, j'ai procédé à une phase de validation technique en local. Ces tests m'ont permis de vérifier l'intégration du script dans l'interface Chutney et de confirmer que les messages d'erreurs s'affichaient correctement selon les cas (Succès, Échec ou Rejet).

1. Cas Nominal (Succès)

step description — en 6ms le 1/27/26, 3:50 PM

sql-with-validation on **db_chutney** - file://aa

Entrées

statements

```
[
select * from scenario
]
```

validations

```
{
  "DESCRIPTION": "aaa",
  "ID": 1,
  "TITLE": "test"
}
```

Journaux

```
| ID | TITLE          | DESCRIPTION          | CONTENT
|----|----------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
| 1  | scenario title | scenario description | {"givens":[{"description":"step description","implementation":{"type":"sql-with-validation","target":"db_chutney","inputs":{"statements":["select * from scen
Validation [ID] - attendue: 1
Validation [DESCRIPTION] - actuelle: scenario description, attendue: aaa
Validation [TITLE] - actuelle: scenario title, attendue: test
```

3. Cas d'Erreur (Rejet Fonctionnel)

step description — en 61ms le 1/27/26, 3:50 PM

sql-with-validation on **db_chutney** - file://aa

Entrées

statements

```
[
select * from scenario
]
```

validations

```
{
  "ID": 1,
  "TITLE": "scenario title"
}
```

Journaux

```
| ID | TITLE          | DESCRIPTION          | CONTENT
|----|----------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
| 1  | scenario title | scenario description | {"givens":[{"description":"step description","implementation":{"type":"sql-with-validation","target":"db_chutney","inputs":{"statements":["select * from scena
Validation [ID] - attendue: 1
Validation [TITLE] - attendue: scenario title
```

Sorties

3. Cas Limite (Retour Vide)

step description

sql-with-validation on **db_chutney** - file://aa

Entrées

statements

```
[
select * from scenario where id = 3
]
```

validations

```
{
  "DESCRIPTION": "aaa",
  "ID": 1,
  "TITLE": "test"
}
```

Journaux

```
| ID | TITLE | DESCRIPTION | CONTENT | TAGS | CREATION_DATE | DATASET | DEFAULT_DATASET_ID | ACTIVATED | USER_ID | UPDATE_DATE | VERSION |
|----|-------|-------------|---------|------|---------------|---------|--------------------|-----------|---------|-------------|---------|
La requête SQL n'a retourné aucune ligne
```

6. LISTAGE ET ANALYSE DES ERREURS

6.1 Test sur Chutney en local

Dans un second temps, j'ai réalisé un recensement sur Excel des scripts générant des messages d'erreur incompréhensibles, et ce pour plusieurs **campagnes de tests (ensembles de scripts automatisés exécutés à la suite).**

Ce tableau d'analyse comportait 5 colonnes : le nom du scénario, le message d'erreur brut, le type d'erreur, une proposition de correctif et la solution retenue. J'ai classifié les incidents selon trois typologies :

- **Régression :** Une valeur ou un résultat inattendu qui interrompt le test.
- **JDD (Jeu de Données) :** Une incohérence liée aux données injectées.
- **Environnement :** Un problème d'infrastructure (ex: base de données inaccessible ou conflit avec d'autres tests).

Ce travail de classification a été essentiel pour ma compréhension du système. Il m'a appris à identifier et analyser finement les causes d'échec, car un simple détail technique peut changer totalement la nature de l'erreur.

6.2 Analyse d'un cas concret

```
statements
[ SELECT dem.DMD_ID_DEMANDE as ID_DEMANDE, dem.DMD_ID_EXTERNE, dem.DMD_DATE_RECEPTION, demu.DMU_ID_OBJET,
demu.DMU_TYPE_OBJET, demu.DMU_STATUT AS DMU_STATUT FROM rds_demande dem LEFT JOIN rds_demande_unitaire
demu ON demu.dmd_id_demande = dem.dmd_id_demande WHERE dem.DMD_DATE_RECEPTION > timestamp '2025-11-20
10:38:45' AND demu.DMU_ID_OBJET = '69388R8561-22271' AND dem.DMD_SERVICE = 'SMC29' ORDER BY
dem.DMD_DATE_RECEPTION DESC FETCH FIRST ROW ONLY ]
                                        Journaux
| ID_DEMANDE | DMD_ID_EXTERNE | DMD_DATE_RECEPTION | DMU_ID_OBJET | DMU_TYPE_OBJET | DMU_STATUT | |-------
-----|----------------|--------------------|--------------|----------------|------------|
Validation [size > 0 : ${#recordResult.get(0).count > 0}] : KO
                                        Sorties
```

De nombreux scénarios présentaient cette erreur. Au départ, je pensais que c'était lié au JDD (Jeu de Données) car le tableau de résultats était vide. Cependant, en vérifiant d'autres exécutions du script où la requête était identique et fonctionnait sans problème, j'ai pu écarter cette piste. J'en ai conclu qu'il s'agissait finalement d'un problème **environnemental** : le script n'a pas pu s'exécuter correctement car l'environnement cible était déjà sollicité par un autre processus.

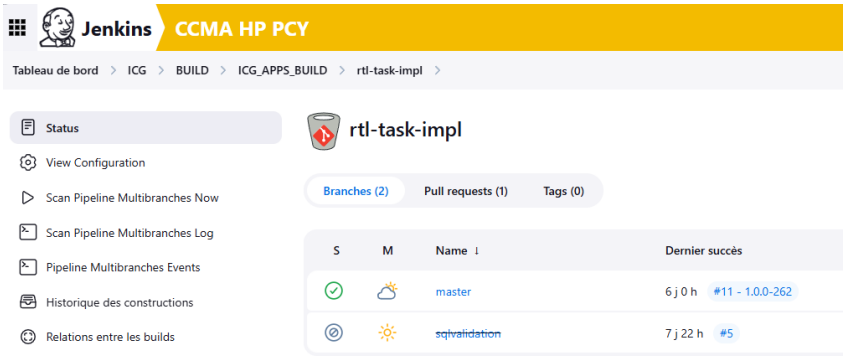
7. DÉPLOIEMENT ET INTÉGRATION CONTINUE (CI/CD)

Le déploiement du script vers les environnements de recette suit un processus automatisé strict via **Jenkins**. Ce pipeline "CI/CD" (Intégration Continue / Déploiement Continu) garantit que le code est validé, versionné et compatible avec l'architecture globale d'Enedis avant d'être installé.

Le processus se décompose en 4 étapes clés :

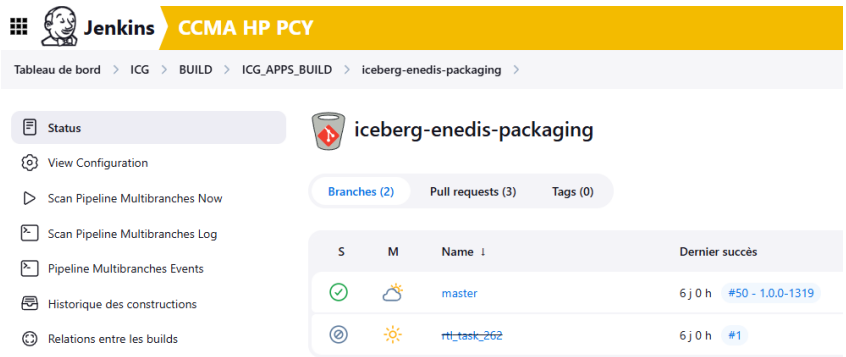
ETAPE 1 : La Compilation (Build)

Tout commence par le code source Java. Sur le projet rtl-task-impl, j'ai travaillé sur la branche **sqlvalidation**. Cette étape compile mon code pour vérifier qu'il ne contient aucune erreur technique avant d'aller plus loin.



ETAPE 2 : Le Packaging (Assemblage)

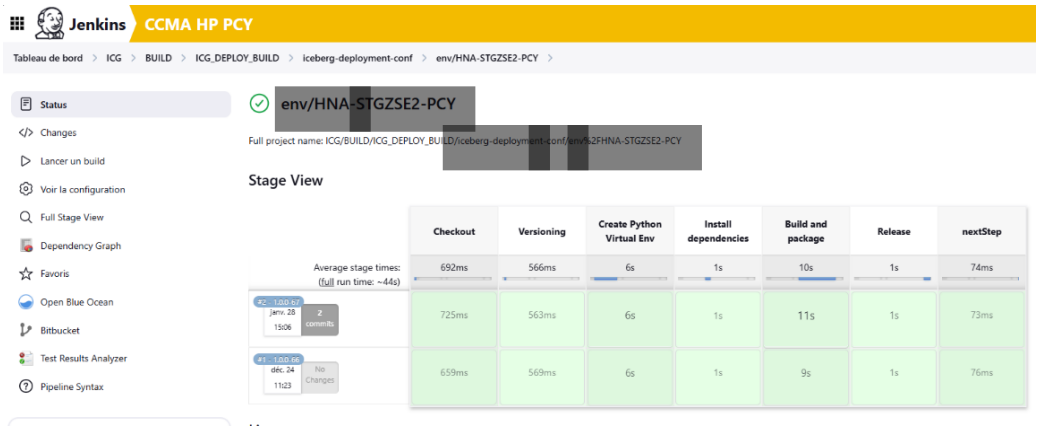
Le code compilé est ensuite récupéré par le job iceberg-enedis-packaging. Ce pipeline rassemble les différents modules (dont le mien, via la branche rtl_task_262) pour créer un **artefact unique** (un "paquet" logiciel complet), prêt à être distribué.



ETAPE 3 : La Configuration de l'Environnement

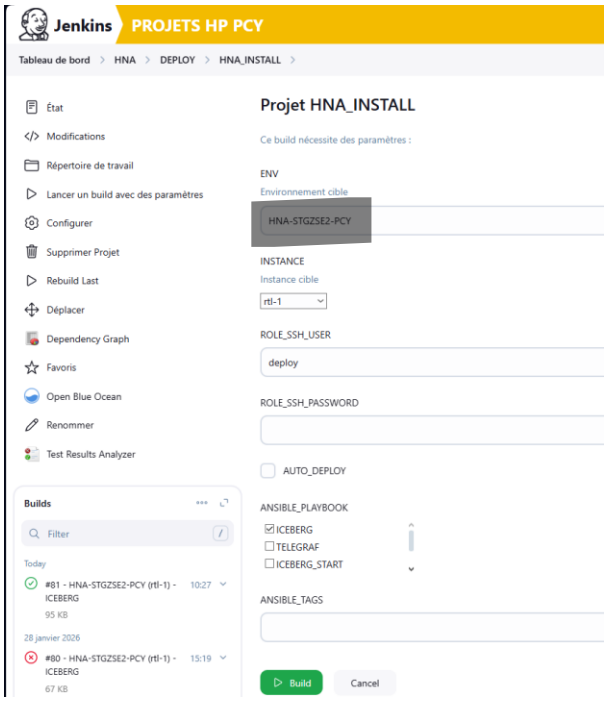
Avant de pouvoir installer ce paquet, il faut préparer le terrain. C'est le rôle du pipeline iceberg-deployment-conf. Comme on le voit sur la "Stage View", ce job prépare la configuration spécifique pour l'environnement cible. Il exécute plusieurs tâches critiques automatiquement :

- **Versioning :** Il fige la version de la configuration.
- **Install dependencies :** Il installe les dépendances chutney nécessaires.
- **Build and package :** Il génère le paquet de configuration prêt à l'emploi.



ETAPE 4 : Le Déploiement Final (Release)

Enfin, le déploiement est déclenché via le job HNA_INSTALL. Ce formulaire permet de lancer l'installation sur le serveur en utilisant un playbook **Ansible** (ICEBERG). C'est cette action qui dépose le script et redémarre les services pour que la nouvelle fonctionnalité soit active.



8. INTÉGRATION ET INDUSTRIALISATION (PROJET RTL-AT)

Une fois la librairie Java déployée via Jenkins, elle devient disponible pour le framework d'automatisation global (rtl-at). Cette étape consiste à rendre la nouvelle fonctionnalité accessible aux testeurs via le code Kotlin.

8.1 L'adaptation en Kotlin

```kotlin
fun ChutneyStepBuilder.SqlWithValidationAction(  3 Usages  Clement BLANIC *
    target: String,
    statements: List<String>,
    validations: Map<String, String>,
    outputs: Map<String, String>
) {
    implementation = ChutneyStepImpl(
        type = "sql-with-validation",
        target = target.toString(),
        inputs = mapOf(
            "statements" to statements,
            "validations" to validations,
        ),
        outputs = outputs
    )
}
```

J'ai implémenté la fonction SqlWithValidationAction dans le projet principal rtl-at. Ce code sert de "pont" : il permet de transformer les appels simplifiés du framework Kotlin en objets Java complexes que mon module peut comprendre.

8.2 Déploiement massif (Le fichier sql.kt)

```kotlin
Step( description = "${prefix}La demande est traitée",  strategy = RetryTimeOutStrategy(Timeout,  retryDelay = retry)) {
    SqlWithValidationAction(
        target = DB_RDS,
        statements = listOf(sqlStatement),
        validations = mapOf("DMU_STATUT" to "SUCCES,ECHEC,REJET"),
        outputs = mapOf(
            "EXTRACTED_DMU_DM" to "recordResult.get(0)".spEL,
            "DMD_ID_EXTERNE" to "firstRow.get('DMD_ID_EXTERNE')".spEL,
            "idDemande" to "firstRow.get('ID_DEMANDE')".spEL
        )
    )
}
```

J'ai ensuite appliqué cette fonction au fichier central sql.kt, ensuite je l'ai commit sur BitBucket. L'impact est majeur : cette modification permet de mettre à jour **plus de 100 scripts automatisés** d'un seul coup. Au lieu de devoir repasser sur chaque test manuellement, ils héritent tous automatiquement de la nouvelle validation.

8.3 Résultat final en production

```
Retry strategy definition : [timeOut 20 min] [delay 10 s]
Try number : 2
| ID_DEMANDE | DMD_ID_EXTERNE | DMD_DATE_RECEPTION | DMU_ID_OBJET | DMU_TYPE_OBJET | DMU_STATUT |
DMU_CODE_RETOUR | |---------------------|-----------------------------------------------|---------------
----------|------------------|----------------|------------|-----------------| | 1100000000292128303 |
ICB_SMC13_02_PING_K_V2_CC2_1770285365754_710821 | 2026-02-05 10:56:05.813 | 69389R9137-00812 | PDK |
SUCCES | null |
Validation [DMU_STATUT] - attendue: SUCCES,ECHEC,REJET
Error(s) on last step execution:
La requête SQL n'a retourné aucune ligne
Sorties
```

Lors de l'exécution des campagnes de test, le changement est immédiat. Un message est désormais transmis en clair en cas d'erreur ou même quand le test passe. Comme on le voit ci-dessus : *"La requête SQL n'a retourné aucune ligne"* pour l'exécution juste avant puis « Validation [DMU_STATUT] – attendue : SUCCES,ECHEC,REJET » pour la dernière exécution en réussite. On a donc éliminé l'affichage technique obscur au profit d'une information compréhensible par tout le monde.

Conclusion

Au terme de ce stage, je tire un bilan très positif de mon expérience au sein d'Enedis.

D'une part, mon travail a permis de contribuer concrètement aux campagnes d'automatisation du projet Linky. J'ai conçu et développé des modules techniques (notamment l'action de validation SQL), qui ont vocation à être intégrés dans les processus quotidiens pour automatiser et fiabiliser les tâches de vérification du support.

D'autre part, ce stage m'a offert une riche expérience de développement. J'ai pu progresser significativement dans l'utilisation d'outils et langages clés : **Java** (avec la gestion des Tests Unitaires), **Chutney**, **SQL** via DBeaver, ainsi que les pipelines d'intégration **Jenkins**. Ces compétences techniques ont été renforcées par la nécessité de comprendre les systèmes existants, ainsi que de relire et réutiliser du code déjà écrit.

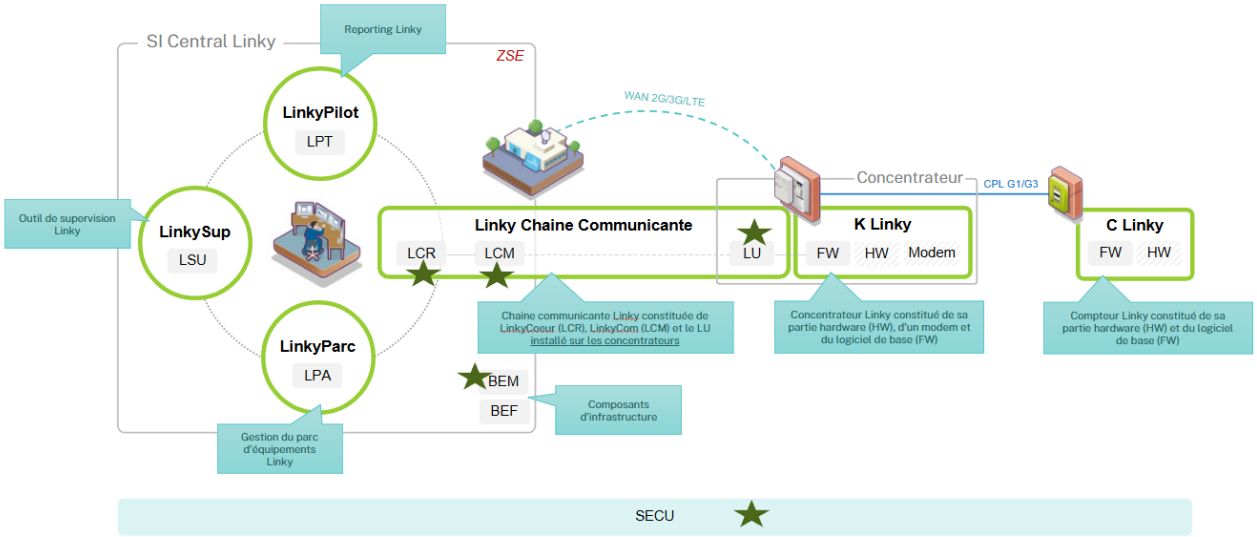
Par ailleurs, j'ai eu l'occasion de me former à la cybersécurité en entreprise, grâce à des formations spécifiques proposées en interne. Celles-ci m'ont sensibilisé à des aspects essentiels comme le respect de la charte informatique, les bonnes pratiques de sécurité (verrouillage de session, robustesse des mots de passe, gestion des accès VPN), et la protection des données sensibles.

Enfin, ce stage m'a permis de m'intégrer pleinement à un environnement professionnel, de comprendre les modes de communication au sein d'une équipe, et de mesurer la complexité des projets menés en collaboration. J'ai appris à interagir efficacement avec mes collègues, en présentiel ou via **Teams**, afin de résoudre des problèmes techniques ou d'obtenir des informations cruciales. J'ai également découvert les enjeux de la coordination et du suivi dans un projet impliquant plusieurs intervenants, en participant activement aux réunions d'équipe.

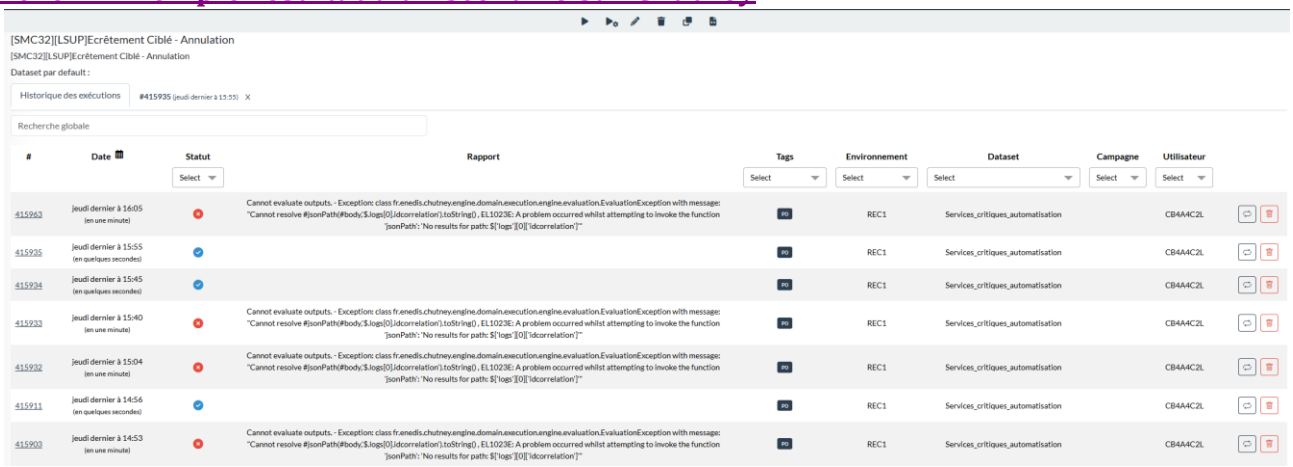
En conclusion, cette expérience m'a beaucoup apporté sur les plans technique, organisationnel et humain. Elle m'a conforté dans mon envie de poursuivre dans le domaine du développement applicatif, en me donnant un aperçu concret des défis passionnants rencontrés dans le monde professionnel.

Annexes

Annexe 1 : Schéma du système Informatique



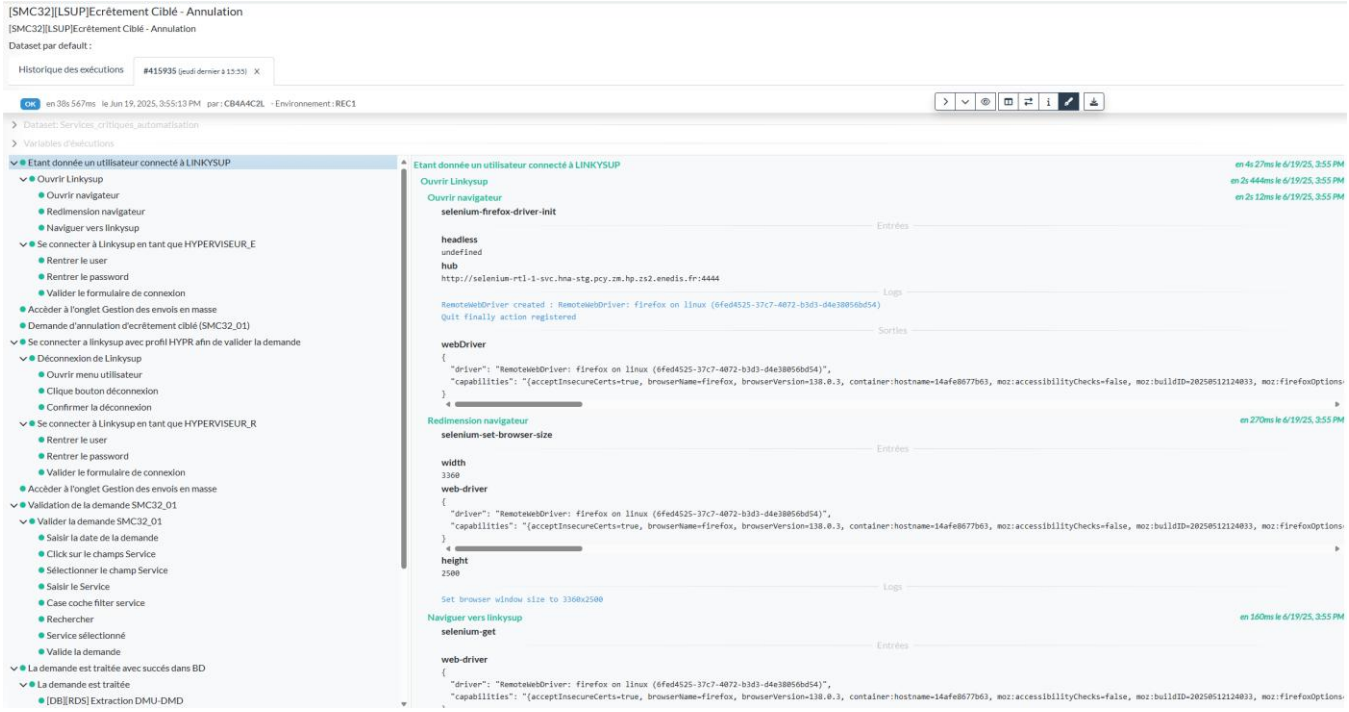
Annexe 2 : Exemple résultat d'un scenario sur Chutney



Sur ce premier écran de **Chutney**, nous pouvons visualiser toutes les exécutions du scénario **SMC32**. Plusieurs informations y sont affichées pour chaque exécution :

- **L'ID** de l'exécution,
- **La date** et l'heure d'exécution,
- **Le statut** : une croix rouge indique une erreur survenue pendant l'exécution, un rond bleu signifie que le script est valide et s'est déroulé correctement, et un rond gris montre que le script a été interrompu manuellement avant la fin,
- **Les tags** qui permettent de trier ou filtrer les scripts selon différents critères,

- **L'environnement de simulation** utilisé pour l'exécution ; il en existe trois : **REC1**, **REC2** et **REC3**,
- **La base de données** sur laquelle le scénario s'appuie,
- **La campagne** si le scénario fait partie d'un ensemble de tests automatisés,
- Et enfin **l'utilisateur** qui a lancé le scénario.



Et sur ce second screen, on peut voir en détail le scénario avec toutes les étapes exécutées. Une pastille de couleur (rouge ou verte) indique si une erreur est survenue ou non. On retrouve aussi des détails comme des ID, des actions web et d'autres informations.